

Universidad Nacional de General Sarmiento

Instituto de Industria - Licenciatura en Sistemas

SISTEMAS OPERATIVOS Y REDES II

Trabajo Práctico 2

Seguridad en Sistemas Operativos



Integrantes: Cabezas Natalia, Faccini Gonzalo, Lucero Esteban, Mendoza Franco, Montoro Rodrigo, Zárate Santiago

Grupo: 04

Comisión: 01

Fecha de entrega: 07/10/2026

Profesor: Benjamín Chuquimango

Parte 1 – Hardening.....	3
Pruebas de Ejecución y Evidencias.....	3
Salida de diagnóstico inicial con --check.....	3
Primera ejecución de --apply.....	4
Segunda ejecución de --apply.....	4
Verificación del registro y restauración con --restore.....	4
Análisis e Interpretación Técnica.....	6
Análisis 1 – Control de Seguridad Seleccionado (Restricción de Acceso SSH).....	6
Análisis 2 – Demostración de Idempotencia del Script.....	6
Parte 2 – AppArmor.....	7
Verificación de los permisos DAC.....	7
Confirmación y validación del perfil.....	7
Prueba de perfil en Modo Complain.....	8
Prueba de perfil en Modo Enforce.....	8
Análisis 3 – Diferencia entre DAC y AppArmor.....	9
Análisis 4 – Diferencia entre Complain y Enforce.....	9
Parte 3 – Auditoría, Syscall e Integridad.....	10
Reglas de auditd.....	10
Correlación entre strace y auditd.....	12
Configuración de AIDE.....	13
Escenario Integrador.....	15
Reconstrucción de los eventos.....	15

Introducción

En el presente trabajo se abordó la administración de la seguridad de un sistema operativo GNU/Linux mediante diferentes herramientas, con el objetivo de trabajar sobre distintas capas de protección. Para esto se aplicó el principio de Defensa en Profundidad, buscando que, en caso de que uno de los mecanismos de seguridad falle o sea vulnerado, existan otras medidas que permitan reducir el riesgo o registrar lo ocurrido.

El trabajo se realizó sobre Ubuntu Server 24.04 LTS y se organizó en tres perspectivas principales.

1. **Hardening y reducción de la superficie de ataque:** Modificar configuraciones por defecto que resultan innecesariamente riesgosas (como accesos remotos como root o permisos de creación laxos) y automatizar estas correcciones mediante scripts idempotentes.
2. **Control de Acceso Obligatorio (MAC):** Se utilizó AppArmor para restringir las acciones que puede realizar una aplicación determinada. De esta manera, se buscó que el programa pudiera acceder únicamente a los recursos necesarios para su funcionamiento, incluso cuando los permisos tradicionales del sistema de archivos (DAC) permitieran un acceso más amplio.
3. **Auditoría e Integridad de Archivos:** Se utilizaron auditd y AIDE para observar y detectar cambios en el sistema. AUDITD se utilizó para establecer una línea base y posteriormente detectar modificaciones en los archivos controlados.

A continuación en las correspondientes secciones pasaremos a analizar y profundizar sobre estos aspectos para dar un mayor hincapié.

Parte 1 – Hardening

En esta sección se trabajó sobre el hardening del sistema, es decir, sobre la aplicación de distintas configuraciones destinadas a reducir riesgos de seguridad en Ubuntu Server. Para realizar estas modificaciones se utilizó el script `hardening.sh`, que permite consultar el estado actual de las configuraciones, aplicar los cambios necesarios y restaurarlos posteriormente. Además, se verificó que el script pueda ejecutarse más de una vez sin volver a realizar cambios que ya fueron aplicados.

Pruebas de Ejecución y Evidencias

A continuación se incluyen las evidencias de las ejecuciones del script.

Salida de diagnóstico inicial con `--check`

Se ejecutó `./hardening.sh --check` antes de aplicar ningún cambio. Esta opción permite conocer el estado de las configuraciones sin realizar cambios en el sistema.

El objetivo de esta ejecución fue obtener un punto de partida y comprobar cuáles de los controles requeridos todavía no se encontraban configurados.

```
vboxuser@Ubuntu:~/TP2-SOR/codigo_base/hardening$ sudo ./hardening.sh --check
2026-10-05T16:09:10+00:00 [CHECK] [UID0] Solo root posee UID 0
2026-10-05T16:09:10+00:00 [CHECK] [SSH] Fragmento valido y configuracion efectiva con PermitRootLogin no
2026-10-05T16:09:10+00:00 [CHECK] [PWQUALITY] Politica definida y pam_pwquality.so activo
2026-10-05T16:09:10+00:00 [CHECK] [UMASK] Las nuevas sesiones aplicaran umask 027 mediante /etc/profile.d/99-tp2-hardening.sh
2026-10-05T16:09:11+00:00 [CHECK] [SYSCTL] Archivo persistente y valores efectivos fs.protected_* = 1
2026-10-05T16:09:11+00:00 [RESUMEN] [TP2] Aplicados=0 Omitidos=0 Verificados=5 Errores=0
```

La evidencia permite observar el estado inicial de los controles antes de aplicar el hardening.

Primera ejecución de --apply

Se ejecutó `./hardening.sh --apply`. En esta etapa se aplicaron las configuraciones de seguridad que todavía no se encontraban establecidas. El script también registra las acciones realizadas, permitiendo comprobar qué controles fueron modificados y cuál fue el resultado de cada operación.

```
vboxuser@Ubuntu:~/TP2-SOR/codigo_base/hardening$ sudo ./hardening.sh --apply
2026-10-05T16:04:06+00:00 [CHECK] [UID0] Solo root posee UID 0
2026-10-05T16:04:06+00:00 [ERROR] [SSH] No se pudo recargar el servicio SSH; se restauro el archivo
2026-10-05T16:04:06+00:00 [APPLIED] [PWQUALITY] Aplicada politica minlen=12, minclass=3 y maxrepeat=3
2026-10-05T16:04:06+00:00 [APPLIED] [UMASK] Configurado umask 027 para nuevas sesiones
2026-10-05T16:04:06+00:00 [APPLIED] [SYSCTL] Persistidos fs.protected_hardlinks=1 y fs.protected_symlinks=1
2026-10-05T16:04:06+00:00 [RESUMEN] [TP2] Aplicados=3 Omitidos=0 Verificados=1 Errores=1
```

Los bloque referentes a PWQUALITY, UMASK, SYSCTL se ejecutaron de forma correcta, quedando marcados con la etiqueta [APPLIED].

El control de SSH reportó un error. Esto se debió a que el SSH no se encontraba en ejecución en la máquina virtual al momento de aplicar los cambios.

Segunda ejecución de --apply

Tras iniciar la pila del servicio SSH mediante el comando `sudo systemctl start ssh`, se procede a ejecutar por segunda vez consecutiva el comando `./hardening.sh --apply` para validar la idempotencia del script.

```
vboxuser@Ubuntu:~/TP2-SOR/codigo_base/hardening$ sudo systemctl start ssh
vboxuser@Ubuntu:~/TP2-SOR/codigo_base/hardening$ sudo ./hardening.sh --apply
2026-10-05T16:06:47+00:00 [CHECK] [UID0] Solo root posee UID 0
2026-10-05T16:06:47+00:00 [APPLIED] [SSH] Aplicado PermitRootLogin no en /etc/ssh/sshd_config.d/99-tp2-hardening.conf
2026-10-05T16:06:47+00:00 [SKIPPED] [PWQUALITY] La politica ya coincide con /etc/security/pwquality.conf.d/99-tp2-hardening.conf
2026-10-05T16:06:47+00:00 [SKIPPED] [UMASK] El fragmento ya configura umask 027
2026-10-05T16:06:47+00:00 [SKIPPED] [SYSCTL] El archivo y los valores efectivos ya son correctos
2026-10-05T16:06:47+00:00 [RESUMEN] [TP2] Aplicados=1 Omitidos=3 Verificados=1 Errores=0
```

Al volver a correr el script con el servicio activo, se pudo aplicar correctamente y el sistema pasó al estado seguro. Luego de la reejecución, el script evaluó el estado del sistema y determinó que las configuraciones ya se encontraban presentes y activas.

Verificación del registro y restauración con --restore

Se verifica el funcionamiento de los mecanismos de restauración y comprobación implementados en `hardening.sh`. Primero, mediante la opción `--restore`, se comprueba que el sistema pueda recuperar el estado registrado antes de aplicar las medidas de hardening.

Luego, con la opción --check, se realiza una auditoría final para verificar que la configuración actual coincida con el estado esperado y detectar posibles inconsistencias.

```
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo ./hardening/hardening.sh --restore
2026-10-05T15:08:41+00:00 [CHECK] [UID0] Solo root posee UID 0
2026-10-05T15:08:41+00:00 [APPLIED] [RESTORE] Eliminado /etc/ssh/sshd_config.d/99-tp2-hardening.conf porque no existia antes del TP
2026-10-05T15:08:41+00:00 [APPLIED] [RESTORE] Eliminado /etc/security/pwquality.conf.d/99-tp2-hardening.conf porque no existia antes del TP
2026-10-05T15:08:41+00:00 [APPLIED] [RESTORE] Eliminado /etc/profile.d/99-tp2-hardening.sh porque no existia antes del TP
2026-10-05T15:08:41+00:00 [APPLIED] [RESTORE] Eliminado /etc/sysctl.d/99-tp2-hardening.conf porque no existia antes del TP
2026-10-05T15:08:41+00:00 [RESUMEN] [TP2] Aplicados=4 Omitidos=0 Verificados=1 Errores=0
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo ./hardening/hardening.sh --check
2026-10-05T15:08:46+00:00 [CHECK] [UID0] Solo root posee UID 0
2026-10-05T15:08:46+00:00 [ERROR] [SSH] Falta el fragmento /etc/ssh/sshd_config.d/99-tp2-hardening.conf o no fija PermitRootLogin no
2026-10-05T15:08:46+00:00 [ERROR] [PWQUALITY] Falta /etc/security/pwquality.conf.d/99-tp2-hardening.conf o no coincide con la politica definida
2026-10-05T15:08:46+00:00 [ERROR] [UMASK] Falta /etc/profile.d/99-tp2-hardening.sh o no configura umask 027
2026-10-05T15:08:46+00:00 [ERROR] [SYSCTL] Falta /etc/sysctl.d/99-tp2-hardening.conf o no define los dos enlaces protegidos
2026-10-05T15:08:46+00:00 [RESUMEN] [TP2] Aplicados=0 Omitidos=0 Verificados=1 Errores=4
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$
```

El script eliminó los fragmentos creados en /etc/ssh/sshd_config.d/, /etc/security/pwquality.conf.d/, /etc/profile.d/ y /etc/sysctl.d/. Luego, con ./hardening.sh --check, el sistema reportó Errores=4, confirmando que los controles ya no se encuentran presentes.

Para auditar el historial de ejecuciones, se consultó el archivo de registro persistente del script: cat /var/log/tp2-hardening.log

```
2026-10-05T15:08:26+00:00 [APPLIED] [SSH] Aplicado PermitRootLogin no en /etc/ssh/sshd_config.d/99-tp2-hardening.conf
2026-10-05T15:08:26+00:00 [APPLIED] [PWQUALITY] Aplicada politica minlen=12, minclass=3 y maxrepeat=3
2026-10-05T15:08:26+00:00 [APPLIED] [UMASK] Configurado umask 027 para nuevas sesiones
2026-10-05T15:08:26+00:00 [APPLIED] [SYSCTL] Persistidos fs.protected_hardlinks=1 y fs.protected_symlinks=1
2026-10-05T15:08:26+00:00 [RESUMEN] [TP2] Aplicados=4 Omitidos=0 Verificados=1 Errores=0
2026-10-05T15:08:29+00:00 [CHECK] [UID0] Solo root posee UID 0
2026-10-05T15:08:29+00:00 [SKIPPED] [SSH] El fragmento ya fija PermitRootLogin no
2026-10-05T15:08:29+00:00 [SKIPPED] [PWQUALITY] La politica ya coincide con /etc/security/pwquality.conf.d/99-tp2-hardening.conf
2026-10-05T15:08:29+00:00 [SKIPPED] [UMASK] El fragmento ya configura umask 027
2026-10-05T15:08:29+00:00 [SKIPPED] [SYSCTL] El archivo y los valores efectivos ya son correctos
2026-10-05T15:08:29+00:00 [RESUMEN] [TP2] Aplicados=0 Omitidos=4 Verificados=1 Errores=0
2026-10-05T15:08:41+00:00 [CHECK] [UID0] Solo root posee UID 0
2026-10-05T15:08:41+00:00 [APPLIED] [RESTORE] Eliminado /etc/ssh/sshd_config.d/99-tp2-hardening.conf porque no existia antes del TP
2026-10-05T15:08:41+00:00 [APPLIED] [RESTORE] Eliminado /etc/security/pwquality.conf.d/99-tp2-hardening.conf porque no existia antes del TP
2026-10-05T15:08:41+00:00 [APPLIED] [RESTORE] Eliminado /etc/profile.d/99-tp2-hardening.sh porque no existia antes del TP
2026-10-05T15:08:41+00:00 [APPLIED] [RESTORE] Eliminado /etc/sysctl.d/99-tp2-hardening.conf porque no existia antes del TP
2026-10-05T15:08:41+00:00 [RESUMEN] [TP2] Aplicados=4 Omitidos=0 Verificados=1 Errores=0
2026-10-05T15:08:46+00:00 [CHECK] [UID0] Solo root posee UID 0
2026-10-05T15:08:46+00:00 [ERROR] [SSH] Falta el fragmento /etc/ssh/sshd_config.d/99-tp2-hardening.conf o no fija PermitRootLogin no
2026-10-05T15:08:46+00:00 [ERROR] [PWQUALITY] Falta /etc/security/pwquality.conf.d/99-tp2-hardening.conf o no coincide con la politica definida
2026-10-05T15:08:46+00:00 [ERROR] [UMASK] Falta /etc/profile.d/99-tp2-hardening.sh o no configura umask 027
2026-10-05T15:08:46+00:00 [ERROR] [SYSCTL] Falta /etc/sysctl.d/99-tp2-hardening.conf o no define los dos enlaces protegidos
2026-10-05T15:08:46+00:00 [RESUMEN] [TP2] Aplicados=0 Omitidos=0 Verificados=1 Errores=4
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$
```

Esto muestra que el archivo registra cronológicamente las marcas de tiempo de cada acción. Se observan los eventos [APPLIED] [RESTORE] confirmando la eliminación de cada archivo, garantizando la auditabilidad del proceso.

Análisis e Interpretación Técnica

Se analizan dos aspectos principales del script `hardening.sh`. En primer lugar, seleccionamos uno de los controles de seguridad aplicados para poder explicar qué riesgo busca reducir y cómo se verificó que el cambio haya sido aplicado correctamente. En segundo lugar, se analiza la idempotencia del script a partir de la comparación entre su primera y segunda ejecución.

Análisis 1 – Control de Seguridad Seleccionado (Restricción de Acceso SSH)

Uno de los controles consiste en impedir que la cuenta `root` pueda iniciar sesión directamente mediante SSH. Esto se realizó mediante la creación de un archivo de configuración `/etc/ssh/sshd_config.d/99-tp2-hardening.conf` con la configuración `PermitRootLogin no`.

La cuenta `root` dispone de privilegios administrativos sobre todo el sistema. Permitir su acceso directo mediante SSH implica que un atacante podría intentar obtener acceso remoto utilizando directamente una cuenta con privilegios elevados. Entonces, la configuración `PermitRootLogin no` impide este tipo de acceso directo. De esta manera, el acceso remoto debe realizarse mediante una cuenta de usuario normal y, posteriormente mediante `sudo`. Esto permite además identificar qué usuario realizó una determinada acción administrativa.

Para comprobar su efecto en el sistema se realizaron las siguientes verificaciones:

1. Verificar archivo de configuración: Se comprobó que existiera `/etc/ssh/sshd_config.d/99-tp2-hardening.conf` y que contuviera la directiva `PermitRootLogin no`.
2. Validación: Se ejecutó `sudo ssh -t`, para comprobar que la configuración de SSH no contenga errores.
3. Comprobación de la configuración: Se ejecutó `sshd -T | grep -i permitrootlogin`. El resultado mostró `permitrootlogin no`, confirmando que la configuración aplicada por el script es reconocida por el servicio SSH.

Análisis 2 – Demostración de Idempotencia del Script

Cuando volvemos a ejecutar `hardening.sh --apply` sobre un sistema que ya fue configurado no debe volver aplicar innecesariamente las mismas modificaciones, esto se debe a la idempotencia que es la propiedad que permite ejecutar una misma operación más de una vez sin producir cambios adicionales una vez alcanzado el estado deseado.

Para comprobar esta propiedad quedó demostrado en la sección de [Pruebas de Ejecución y Evidencias](#), donde la primera ejecución con `sudo ./hardening.sh --apply` el script encontró que las configuraciones de `hardening` todavía no se encontraban aplicadas, entonces se procedió a realizar los cambios correspondientes. Por eso su salida se registró `[APPLIED]`. En cuanto a la segunda ejecución del mismo comando, los registros fueron registrados como `[SKIPPED]`, ya que las configuraciones requeridas ya se encontraban aplicadas.

Esto demuestra que el script reconoce que el sistema ya se encuentra en el estado esperado y evita realizar nuevamente las mismas modificaciones.

Parte 2 – AppArmor

En esta sección se trabajó con AppArmor como mecanismo de Control de Acceso Obligatorio (MAC). El objetivo es establecer una política para el programa `/usr/local/bin/tp2-reader`, de manera que pueda acceder al archivo `publico.txt`, pero no pueda leer `confidencial.txt`.

Para realizar la prueba utilizamos los archivos ubicados en `/srv/tp2/datos/`. Primero comprobamos que los permisos tradicionales del sistema de archivos (DAC) permiten la lectura de ambos archivos. Luego se aplica el perfil de AppArmor y se comprueba cómo cambia el comportamiento del programa.

Verificación de los permisos DAC

Antes de aplicar el perfil de AppArmor verificamos los permisos y propietarios de `publico.txt` y `confidencial.txt`. Esta comprobación permite establecer el estado inicial de los archivos y verificar que los permisos tradicionales permiten su lectura. Mediante esto podemos diferenciar posteriormente el efecto de los permisos DAC respecto de la restricción aplicada por AppArmor.

```
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ ls -l /srv/tp2/datos/
total 12
-rw-r--r-- 1 root root 58 oct 5 15:05 confidencial.txt
-rw-r--r-- 1 root root 155 oct 7 14:13 eventos.log
-rw-r--r-- 1 root root 92 oct 7 14:13 publico.txt
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$
```

Como podemos observar ambos archivos cuentan con permisos de lectura para el usuario y su grupo (`rw-r--r--` / 644).

Luego, se ejecutó el programa `tp2-reader` sobre ambos archivos antes de aplicar el perfil de AppArmor para observar que se realizó correctamente.

```
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo /usr/local/bin/tp2-reader /srv/tp2/datos/publico.txt
Este archivo puede ser leído por la aplicación de prueba.
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo /usr/local/bin/tp2-reader /srv/tp2/datos/confidencial.txt
RECURSO CONTROLADO DEL TP2. No contiene información real.
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$
```

Con esto demostramos que el programa logró abrir y mostrar el contenido de ambos archivos sin restricciones, ya que bajo el esquema de Linux basado en DAC posee acceso legítimo a ambos recursos.

Confirmación y validación del perfil

A continuación se completó el archivo base `usr.local.bin.tp2-reader.base` y lo copiamos al directorio `/etc/apparmor.d/` con el nombre correspondiente al ejecutable. Antes de cargar el

perfil se verificó su sintaxis mediante `apparmor_parser`. Luego se cargó el perfil en el sistema para comenzar con las pruebas de acceso.

```
admin@ubuntu-server:~/TP2-SOR/codigo_base$ sudo cp ./apparmor/usr.local.bin.tp2-reader /etc/apparmor.d/usr.local.bin.tp2-reader
admin@ubuntu-server:~/TP2-SOR/codigo_base$ sudo apparmor_parser -Q /etc/apparmor.d/usr.local.bin.tp2-reader
admin@ubuntu-server:~/TP2-SOR/codigo_base$ sudo apparmor_parser -r /etc/apparmor.d/usr.local.bin.tp2-reader
admin@ubuntu-server:~/TP2-SOR/codigo_base$
```

Esta validación previa nos permite comprobar que el perfil no contiene errores antes de utilizarlo en las pruebas.

Prueba de perfil en Modo Complain

Luego de cargar el perfil, se configuró `/usr/local/bin/tp2-reader` en modo complain. AppArmor registra las operaciones que no están permitidas por el perfil, pero no las bloquea. Esto nos permite observar los accesos realizados por el programa antes de aplicar la restricción de manera efectiva.

Posteriormente, se consultó el registro de auditoría para verificar los eventos generados por AppArmor. En el caso de `confidencial.txt`, se observa un evento `DENIED` asociado a la operación de lectura solicitada por `tp2-reader`. También aparece un evento `ALLOWED`, lo que permite comprobar que, aunque el acceso fue identificado por la política, en modo complain la operación no fue bloqueada.

```
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo aa-complain /usr/local/bin/tp2-reader
[sudo] contraseña para vboxuser:
Estableciendo /usr/local/bin/tp2-reader al modo reclamar.
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo /usr/local/bin/tp2-reader /srv/tp2/datos/publico.txt
Este archivo puede ser leído por la aplicación de prueba.
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo /usr/local/bin/tp2-reader /srv/tp2/datos/confidencial.txt
RECURSO CONTROLADO DEL TP2. No contiene información real.
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo grep -F 'apparmor' /var/log/audit/audit.log | tail -2
type=AVC msg=audit(1791386981.117:930): apparmor="ALLOWED" operation="file_perm" class="file" profile="/usr/local/bin/tp2-reader" name="/srv/tp2/datos/confidencial.txt" pid=29798 comm="tp2-reader" requested_mask="r" denied_mask="r" fsuid=0 ouid=0FSUID="root" OUID="root"
type=AVC msg=audit(1791386981.117:931): apparmor="ALLOWED" operation="file_perm" class="file" profile="/usr/local/bin/tp2-reader" name="/srv/tp2/datos/confidencial.txt" pid=29798 comm="tp2-reader" requested_mask="r" denied_mask="r" fsuid=0 ouid=0FSUID="root" OUID="root"
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$
```

Con esto comprobamos el comportamiento del modo complain ya que AppArmor detecta y registra el acceso que no está contemplado por la política, pero permite que el programa continúe ejecutando la operación. En este caso, `confidencial.txt` pudo ser leído a pesar de que el perfil no concede ese acceso.

Prueba de perfil en Modo Enforce

Finalmente, se cambió el estado del perfil a modo enforce ejecutando `sudo aa-enforce /usr/local/bin/tp2-reader` y se volvió a probar la ejecución sobre ambos archivos.

```
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo aa-enforce /usr/local/bin/tp2-reader
Estableciendo /usr/local/bin/tp2-reader al modo forzar.
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo /usr/local/bin/tp2-reader /srv/tp2/datos/publico.txt
Este archivo puede ser leído por la aplicación de prueba.
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo /usr/local/bin/tp2-reader /srv/tp2/datos/confidencial.txt
tp2-reader: no se pudo abrir '/srv/tp2/datos/confidencial.txt': Permission denied
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo grep -F 'apparmor' /var/log/audit/audit.log | tail -1
type=AVC msg=audit(1791385998.685:872): apparmor="DENIED" operation="open" class="file" profile="/usr/local/bin/tp2-read
er" name="/srv/tp2/datos/confidencial.txt" pid=29691 comm="tp2-reader" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
FSUID="root" OUID="root"
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$
```

Se puede observar que la lectura de `publico.txt` fue permitida, mientras que el acceso a `confidencial.txt` fue rechazado con un mensaje de `Permission denied`. Esto permite comprobar que el perfil de AppArmor está aplicando la restricción definida, que el programa puede acceder al recurso autorizando, pero no al recurso que el perfil no permite.

Análisis 3 – Diferencia entre DAC y AppArmor

Con lo realizado anteriormente permite diferenciar el funcionamiento de los permisos tradicionales del sistema de archivos respecto del control aplicado por AppArmor. Antes de cargar el perfil, `tp2-reader` pudo leer tanto `publico.txt` como `confidencial.txt`. Esto demuestra que los permisos DAC existentes permitían ambas operaciones. Y luego, al activar el perfil AppArmor en modo `enforce`, el programa continuó pudiendo leer `publico.txt`, pero el acceso a `confidencial.txt` fue rechazado.

Por lo tanto, aunque los permisos DAC permitían la lectura del archivo, AppArmor agregó una restricción adicional sobre el proceso y bloqueó el acceso a `confidencial.txt`. Esto demuestra en la práctica el funcionamiento de MAC.

Análisis 4 – Diferencia entre Complain y Enforce

En modo `complain`, el acceso a `confidencial.txt` fue detectado y registrado por AppArmor como un evento `DENIED`, pero la lectura del archivo igualmente se realizó correctamente. Esto demuestra que este modo permite registrar los accesos que no están contemplados por el perfil sin bloquearlos.

En cambio, al utilizar el modo `enforce`, la política pasa a aplicarse de forma efectiva. Por lo tanto, mientras que en `complain` el acceso a `confidencial.txt` fue registrado pero permitido, en `enforce` el mismo acceso es rechazado.

Por lo tanto, la principal diferencia es que `complain` sirve para observar y registrar accesos, mientras que `enforce` aplica las restricciones definidas por el perfil.

Parte 3 – Auditoría, Syscall e Integridad

Reglas de auditd

Para empezar, auditd es un servicio del sistema que corre en segundo plano. Su función es recopilar, registrar y almacenar información detallada sobre los eventos de seguridad que ocurren en el sistema operativo.

Al igual que sshd, normalmente se inicia mediante systemd al arrancar la máquina virtual. La diferencia es que sshd espera conexiones de red, mientras que auditd recibe eventos del subsistema de auditoría del núcleo y los conserva en `/var/log/audit/audit.log`.

Sin embargo, el que auditd esté activo, no significa que registra absolutamente todo. Las reglas cargadas en el núcleo son las que determinan cuáles operaciones se auditan y cuáles no.

En este TP, se definieron dos reglas:

- La primera, registra modificaciones dentro de `/srv/tp2/datos`
- La segunda, audita la ejecución de `/usr/local/bin/tp2-event`.

Estas serán cargadas, en primer lugar, en el archivo de reglas. A continuación, copiamos el contenido que contiene el mismo:

```
-w /srv/tp2/datos -p wa -k tp2_datos  
-a always,exit -F arch=b64 -S execve -F path=/usr/local/bin/tp2-event -k tp2_exec
```

Comencemos analizando la primera regla por partes:

- **-w /srv/tp2/datos (Watch):** Esta parte, establece un punto de vigilancia (también llamado *watch point*) en la ruta especificada. En nuestro caso particular, la ruta corresponde al directorio de datos del trabajo práctico. El sistema vigilará recursivamente cualquier cambio en este directorio.
- **-p wa (Permissions):** Define qué permisos o tipos de operaciones van a ser auditados en la ruta que indicamos con anterioridad:
 - **w (Write):** Registra cualquier operación de escritura realizada en el contenido de los archivos.
 - **a (Attribute change):** Registra todos los cambios realizados en los atributos de los archivos o directorios. Estos podrían ser, por ejemplo, cambios de permisos con `chmod`, modificaciones de propietario con `chown`, o cambios en las marcas de tiempo.
- **-k tp2_datos (Key):** Asigna una key al evento, en este caso, la llamamos `tp2_datos`. Esta etiqueta funciona como un identificador, el cual sirve para facilitar la búsqueda en los logs de auditoría utilizando la herramienta `ausearch`. Esto nos podría servir, por ejemplo, para filtrar rápidamente todos los registros relacionados con esta regla específica.

Mientras que, la segunda regla, tiene la siguiente explicación:

- **-a always,exit:** Esta parte, indica que se debe agregar una regla de auditoría (-a), la cual se activará siempre (always) al finalizar (exit) una llamada al sistema monitoreada.
- **-F arch=b64 (Filter):** Filtra la regla, de forma tal, que aplique únicamente a arquitecturas de 64 bits (b64). Esto asegura que el kernel vigile las llamadas nativas del sistema operativo.
- **-S execve (Syscall):** En esta línea, estamos especificando la llamada al sistema que deseamos interceptar, que es la llamada **execve**. Esta es la llamada estándar en Linux utilizada para ejecutar nuevos programas o binarios.
- **-F path=/usr/local/bin/tp2-event (Filter Path):** Restringe la auditoría exclusivamente a los casos en donde, el binario ejecutado mediante execve, sea el archivo /usr/local/bin/tp2-event.
- **-k tp2_exec (Key):** Al igual que en la regla anterior, etiqueta los registros generados con la key tp2_exec.

Una vez declaradas las reglas, desde la ubicación “codigo_base/”, instalamos el fragmento en el sistema operativo y cargamos las reglas mediante los siguientes comandos:

```
sudo install -m 0640 audit/99-tp2.rules /etc/audit/rules.d/99-tp2.rules
sudo augenrules --load
sudo auditctl -l | grep tp2_
```

Cada comando hace lo siguiente:

- **sudo install -m 0640:** copia el archivo terminado desde el repositorio a /etc/audit/rules.d/, que es una ruta protegida del sistema. Sudo otorga temporalmente permisos administrativos para poder escribir allí; no instala un paquete ni activa por sí solo las reglas. La opción -m 0640 fija sus permisos.
- **sudo augenrules --load:** reúne los archivos *.rules de /etc/audit/rules.d/, genera la configuración consolidada de auditd y carga las reglas en el subsistema de auditoría del núcleo. Sin --load, no se solicita esa carga inmediata.
- **sudo auditctl -l:** consulta y lista las reglas actualmente activas en el núcleo. La opción -l es la letra ele minúscula, no el número 1. | grep tp2_ filtra la salida para mostrar solo las reglas que contienen las claves tp2_datos o tp2_exec. Este último comando consulta; no modifica ninguna regla.

Luego, utilizando el auditctl, obtuvimos el siguiente resultado:

```
santiago in 🌐 sor in TP2-SOR on 📄 main
) sudo auditctl -l | grep tp2_
-w /srv/tp2/datos -p wa -k tp2_datos
-a always,exit -F arch=b64 -S execve -F path=/usr/local/bin/tp2-event -F key=tp2_exec
```

Donde podemos ver que, efectivamente, las reglas fueron cargadas con éxito.

Correlación entre strace y auditd

Strace acompaña una ejecución concreta y sirve para mostrar las llamadas al sistema que realiza el proceso.

En el punto 9.2, ejecutamos el siguiente comando:

```
sudo strace -f -e trace=execve,openat,write \
-o /srv/tp2/evidencias/strace_tp2_event.txt \
/usr/local/bin/tp2-event "evento_grupo_N"
```

A continuación detallamos qué significa cada parte del comando:

- **-f**: sigue también los procesos hijos.
- **-e trace=...**: limita la salida a las llamadas al sistema indicadas.
- **-o**: guarda el resultado en un archivo de evidencia.

```
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo strace -f -e trace=execve,openat,write -o /srv/tp2/evidencias/strace_tp2_event.txt /usr/local/bin/tp2-event "evento_grupo_N"
Evento registrado en /srv/tp2/datos/eventos.log
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ cat /srv/tp2/evidencias/strace_tp2_event.txt
31083 execve("/usr/local/bin/tp2-event", ["/usr/local/bin/tp2-event", "evento_grupo_N"], 0x7ffc52b65700 /* 17 vars */) = 0
31083 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
31083 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
31083 openat(AT_FDCWD, "/srv/tp2/datos/eventos.log", O_WRONLY|O_CREAT|O_APPEND, 0666) = 3
31083 openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = 4
31083 write(3, "2026-10-07T17:38:07+0000 pid=31083...", 64) = 64
31083 write(1, "Evento registrado en /srv/tp2/da...", 48) = 48
31083 +++ exited with 0 +++
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo ausearch -i -k tp2_exec -ts recent | tail -n 1
type=SYSCALL msg=audit(07/10/26 17:38:07.016:1409) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x7ffc52b641c0 a1=0x7ffc52b656e8 a2=0x7ffc52b65700 a3=0x7d0a2b60d0b8 itens=2 ppid=31080 pid=31083 auid=vboxuser uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=pts1 ses=3 comm=tp2-event exe=/usr/local/bin/tp2-event subj=unconfined key=tp2_exec
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$ sudo ausearch -i -k tp2_datos -ts recent | tail -n 1
type=SYSCALL msg=audit(07/10/26 17:38:07.018:1410) : arch=x86_64 syscall=openat success=yes exit=3 a0=AT_FDCWD a1=0x5762f2596019 a2=O_WRONLY|O_CREAT|O_APPEND a3=0x1b6 itens=1 ppid=31080 pid=31083 auid=vboxuser uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=pts1 ses=3 comm=tp2-event exe=/usr/local/bin/tp2-event subj=unconfined key=tp2_datos
vboxuser@ubuntu:~/Escritorio/TP2-SOR/codigo_base$
```

Viendo la salida del strace, podemos sacar las siguientes conclusiones:

- El `execve(...)` = 0 nos indica que la operación fue exitosa.
- `openat(...)` = 3: el núcleo devolvió el descriptor número 3.
- `write(...)` = 64 | `write(...)` = 48: Nos indica que se escribieron 64 y 48 bytes respectivamente.

Luego, tenemos en la misma captura, las salidas correspondientes a la búsqueda con auditd. Veamos la correlación entre los resultados:

- El PID es igual para ambas búsquedas de `ausearch` (31080) y corresponde con el PID que indica `strace` en la línea del `execve`.
- El timestamp muestra que las reglas se aplicaron unos milisegundos después que la ejecución con `strace` (En el `strace` fue: "17:38:07", y en las búsquedas fue "17:38:07.016" y "17:38:07.018").
- El ejecutable es el mismo, si miramos las salidas del `ausearch`, tenemos que `exe=/usr/local/bin/tp2-event` en ambas. Y en la ejecución con `strace` tenemos que el `execve` ejecuta la misma ruta.

- La primera regla que buscamos, encuentra efectivamente la syscall=execve (que es la que debe buscar), mientras que la segunda registra syscall=openat. Ambas llamadas se realizan en las líneas del strace.
- El resultado es igual en ambos. Si miramos la primera salida del ausearch, exit=0, que corresponde a la salida del execve. Luego, si miramos la segunda salida del ausearch, exit=3, lo cual corresponde con el resultado del openat.

Configuración de AIDE

En esta sección vamos a explicar cómo se configuró la herramienta de seguridad AIDE para poder auditar los accesos y modificaciones del directorio /srv/tp2/datos que no fueron deseados.

Para esto lo que se realizó fue la siguiente configuración:

```
database_in=file:/srv/tp2/aide/aide.db  
database_out=file:/srv/tp2/aide/aide.db.new
```

```
report_level=changed_attributes  
report_url=stdout
```

```
TP2_NORMAL = p+ftype+i+n+u+g+s+m+c+sha256
```

```
/srv/tp2/datos TP2_NORMAL
```

En donde database_in y database_out representan donde se van a almacenar la base de datos y donde la primera es fuente de verdad y la segunda registra los cambios en el directorio. Además el report_level en changed_attributes sirve para que AIDE notifique los cambios de manera concisa para que muestre los atributos que fueron cambiados. Luego la configuración report_url=stdout lo que indica es que el reporte se imprima por pantalla y se define a TP2_NORMAL para describir a AIDE que debe monitorear, en nuestro caso estaríamos vigilando los permisos, el tipo de archivo, el número de inodos, el usuario que es dueño del archivo, grupo al que pertenece el archivo, fecha y hora de última modificación, fecha y hora de último cambio de atributos o permisos y el sha256 del contenido del archivo. A partir de esto la última línea define que estas reglas de monitoreo se apliquen al directorio /srv/tp2/datos.

A continuación mostraremos la evidencia donde se muestra que inicializo AIDE, se aplicaron cambios y luego se ejecutó nuevamente la herramienta para visualizar los cambios realizados.

```
santiago in ~ in TP2-SDR/codigo_base/aide on / main
} sudo aide --config=/srv/tp2/config/aide.conf --init
WARNING: /srv/tp2/config/aide.conf:11: special characters in group names are DEPRECATED. Update your config and only use alphanumeric characters (A-Za-z0-9) (Line: 'TP2_NORMAL = p+ftyp
e+i+n+u+g+s+m+c+sha256')
Start timestamp: 2026-10-03 22:24:16 +0000 (AIDE 0.18.6)
AIDE successfully initialized database.
New AIDE database written to /srv/tp2/aide/aide.db.new

Number of entries:      4

-----
The attributes of the (uncompressed) database(s):
-----

/srv/tp2/aide/aide.db.new
MDS       : YQyn84b+0o7jk6Mcf0aMLQ==
SHA1      : MvooQRqF3tDXScuMLEdWZS9A0Vs=
SHA256    : XqFK/rWhd1432+1d8007SRzdHSXY+3V
           UFAfTyaUJZjA=
SHA512    : wp1C0Lg8jVJ78o+Az/4PVDetWShbp1L
           RDyeAGn2RfFWQYKgn9Sq/vVzuSKG1vw
           D2gF6hGyjwmEQc8rVtX8eQ==
RMD160    : R0KAudTz0iPLEJIG8r/qmWJBeLI=
TIGER     : 8RTUXHvb2LTiLGLyEc75kCAy+fVJBLeV
CRC32     : wAcYVQ==
CRC32B    : 0htaaq==
HAVAL     : LRatknPxM/AH9iiPiT4h002TFnhRaI07
           HZFMWedTxog=
WHIRLPOOL : iB2Hb3cVfTRe8AJs/v6w9xXpylkm+se
           JdGne2u0hN0p1rzoTbUmY07D56yt3dW
           ZQGM1eHoahXt1yMz7Rof5g==
GOST      : tUaNB6moo16SD/qLrkY53zrnb4Ww104j
           aJtKvv+Ra1Q=

End timestamp: 2026-10-03 22:24:16 +0000 (run time: 0m 0s)
```

Luego se creó un nuevo archivo llamado prueba_aide.txt, se modificó el archivo de publico.txt y se cambiaron sus permisos del archivo y validamos con AIDE obteniendo lo siguiente:

```
AIDE found differences between database and filesystem!!

Summary:
Total number of entries:      5
Added entries:                1
Removed entries:              0
Changed entries:              2

-----
Added entries:
-----

f++++++: /srv/tp2/datos/prueba_aide.txt

-----
Changed entries:
-----

d = ... mc..      : /srv/tp2/datos
f > p.. mc..H    : /srv/tp2/datos/publico.txt

-----
Detailed information about changes:
-----

Directory: /srv/tp2/datos
Mtime      : 2026-09-20 22:43:51 +0000      | 2026-10-03 22:31:56 +0000
Ctime      : 2026-09-20 22:43:51 +0000      | 2026-10-03 22:31:56 +0000

File: /srv/tp2/datos/publico.txt
Size       : 58                            | 100
Perm       : -rw-r--r--                    | -rw-r-----
Mtime      : 2026-09-20 22:43:51 +0000      | 2026-10-03 22:32:11 +0000
Ctime      : 2026-09-20 22:43:51 +0000      | 2026-10-03 22:32:54 +0000
SHA256     : E2KNVjylpgtXS1KwjRl05dCyF6E8jzHH | uid0cboLV4DR7nqQbkzN75x85eD889aI
           t7vekpZGjsg=                      | 81BedcAPiqa=
```

Aquí AIDE detectó en la primera línea que se creó un archivo nuevo que no estaba almacenado en la base de datos. Luego en las modificaciones marca que el directorio

/srv/datos/ sufrió modificaciones en sus metadatos y por otro lado el archivo publico.txt se sacaron los permisos para otros usuarios y asimismo se cambió el contenido ya que su sh256 cambió y su tamaño aumentó.

La vulnerabilidad de que la base está almacenada en la misma máquina es que es posible que un intruso ingrese con usuario root, realice modificaciones y reinicie la base de datos debido a que esta base tiene permisos de escritura y lectura. Para resolver esto podrían mandar los reportes vía email así el atacante no borra el historial o también se podría almacenar una copia de la base en otro equipo remoto.

Escenario Integrador

Para finalizar, se ejecutó el script escenario_final.sh, con AppArmor en modo enforce, las reglas de auditd estaban cargadas y la base de AIDE había sido inicializada.

Ejecutar escenario nos permitió comprobar de manera conjunta el funcionamiento de las distintas capas de seguridad implementadas.

```
santiago in  sor in TP2-S0R/codigo_base on  main
> sudo ./scripts/escenario_final.sh 2>&1 \
  | sudo tee "$EVIDENCIA/escenario.txt"
=== 1. Lectura publica ===
Este archivo puede ser leído por la aplicacion de prueba.

=== 2. Intento sobre recurso confidencial ===
tp2-reader: no se pudo abrir '/srv/tp2/datos/confidencial.txt': Permission denied

=== 3. Evento controlado ===
Evento registrado en /srv/tp2/datos/eventos.log

=== 4. Cambio controlado de integridad ===
Escenario ejecutado. Reuna ahora evidencia de AppArmor, auditd y AIDE.
```

Reconstrucción de los eventos

En primer lugar, el escenario realizó una lectura sobre el archivo publico.txt mediante tp2-reader. Esta operación fue permitida y el programa pudo acceder correctamente al archivo. Esto coincide con AppArmor configurada anteriormente, que permite a la aplicación realizar la lectura de este recurso.

En segundo lugar, se intentó acceder al archivo confidencial.txt utilizando la misma aplicación. En este caso, la operación fue bloqueada por AppArmor, mostrando el mensaje Permission denied. Esto demuestra que, aunque los permisos tradicionales del archivo permitían la lectura, el perfil de AppArmor en modo enforce impidió el acceso al recurso que no estaba autorizado por la política.

En tercer lugar, el escenario realizó una operación controlada mediante /usr/local/bin/tp2-event. Como resultado, se generó un evento en /srv/tp2/datos/eventos.log. Esta operación se relaciona con las reglas de auditd configuradas para registrar la ejecución de tp2-event y las modificaciones realizadas dentro del directorio /srv/tp2/datos. En la salida

del escenario se observa el mensaje “Evento registrado en /srv/tp2/datos/eventos.log”, lo que permite comprobar que la operación controlada se realizó correctamente.

Por último, el escenario indica la realización de un cambio controlado de integridad. Este cambio debe relacionarse con la línea base creada anteriormente mediante AIDE, que permite comparar el estado actual de los archivos con el estado registrado en la base de referencia.